Course Outline: Capturing Telemetry in Next.js

Title: Capturing Telemetry in Next.js **Skill:** 43 — Collecting telemetry and context-related info from the user and your application **Learnpack:** + Capturar telemetría en Next **File:** s43-w15d43-capturar-telemetria-en-next **Prerequisite:** Sending Telemetry from the Frontend (Skill 43 — the batch sender implementation) **Target Audience:** Beginner developers in a full-stack AI bootcamp building a Next.js application **Total Lessons:** 13 **Format:** Mixed — 8 📖 + 3 💻 + 1 🤔 + 1 outro (23% hands-on)

Course Description

This course answers the question: where exactly in a Next.js application do you hook your telemetry sender? Students implement telemetry capture at three strategic points — the Next.js router for page views, user action handlers for key interactions, and global error handlers plus Web Vitals reporting for automatic monitoring. By the end, students have a complete Next.js telemetry integration that captures all three signal types using the sender built in the previous learnpack.

Course Philosophy

This course emphasizes:

- Hooks over wrappers: use Next.js's built-in hooks (routeChangeComplete, reportWebVitals) rather than wrapping every component
- Passive before active: page views and errors are captured automatically; user actions require explicit placement
- Small surface, big coverage: three well-placed hooks cover 90% of what a production telemetry system needs

---

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Capture Points in Next.js	3
02 - User Action Events	2
03 - Errors and Performance	4
04 - Assessment	2
05 - Outro	1
Total	**13**

00.0 Welcome 📖

Learning Objectives:

- Understand what this course covers: the three capture points in a Next.js application
- Connect this course to the previous (the sender implementation) and the next (guardrails)
- Understand why Next.js has specific hooks that simplify telemetry capture

Content Outline:

- The gap this course fills: you have the sender from the previous course; now you need to know WHERE to call `track()` in a Next.js app
- Three capture categories:
    1. Route changes → page view events (passive, triggered by Next.js router)
    2. User actions → key interaction events (active, attached to specific handlers)
    3. Errors and performance → automatic monitoring signals (passive, via global handlers and reportWebVitals)
- Why Next.js makes some of this easier: `routeChangeComplete` and `reportWebVitals` are built-in hooks that don't require manual instrumentation

Transitions: You built the sender. Now let's wire it up to the three places in your Next.js app that generate the most valuable telemetry signals.

Key Principles:

- Telemetry hooks should be set up once, globally — not duplicated across every page component
- Next.js's App component (`_app.js`) is the best place for router and global error hooks
- The three capture points together cover page navigation, user behavior, and system health

Examples:

- A checkout funnel analysis requires page view events (to see each step) + user action events (checkout_submitted) + error events (checkout_failed) — all three capture points working together

---

01.0 Capture Points in Next.js 🕮

Learning Objectives:

- Identify the three capture categories and their corresponding Next.js hooks
- Understand why `_app.js` is the correct location for global telemetry setup
- Preview the three sections of this course

Content Outline:

- The three capture points and their Next.js hooks:
    1. Router events: `router.events.on('routeChangeComplete', ...)` in `_app.js`
    2. User actions: `onClick`, `onSubmit`, `onChange` handlers on specific components
    3. Errors and performance:
        - `window.onerror` and `window.addEventListener('unhandledrejection', ...)` in `_app.js`

        - React Error Boundary in `_app.js` wrapping the entire app
        - `reportWebVitals(metric)` export from `_app.js`
- Why `_app.js`:
    - It's the root component that wraps every page
    - It runs once per navigation, not once per render
    - `useRouter()` is accessible here for route events
    - `reportWebVitals` is a special export Next.js calls automatically
- The relationship between capture and sending:
    - Capture: detect that an event occurred and call `track(eventName, properties)`
    - Sending: the batch sender from the previous course handles the rest automatically
- Preview: Section 01 covers page views, Section 02 covers user actions, Section 03 covers errors and performance

Transitions: All three capture points fit in or around `_app.js`. Let's start with the one that fires most frequently: page views.

Key Principles:

- Setting up telemetry in `_app.js` means every page automatically gets instrumented — no per-page setup required
- The capture code itself should be minimal: call `track()` with the right event name and a few properties
- Never instrument in individual page components if `_app.js` can do it globally

Examples:

- Without a router event listener in `_app.js`, moving from `/home` to `/checkout` generates no page view event at all
- Adding `window.onerror` in `_app.js` once catches JavaScript errors across your entire application

---

01.1 Tracking Page Views 🕮

Learning Objectives:

- Explain how Next.js client-side routing works and why `routeChangeComplete` is the correct event
- Describe what properties to include in a page_view event
- Understand the difference between initial page load and subsequent client-side navigation

Content Outline:

- Next.js routing and client-side navigation:
    - When a user navigates between pages in Next.js, the URL changes but the page does not fully reload
    - The browser's native `load` event only fires on the initial page load — not on subsequent client-side navigations

    - `router.events` is Next.js's built-in mechanism for listening to these client-side navigations
- The `routeChangeComplete` event:
    - Fires when a route change has fully completed (including data fetching for that page)
    - The event receives the new URL as its argument
    - Alternative events: `routeChangeStart` (route change begins), `routeChangeError` (route change failed)
    - Why `routeChangeComplete` and not `routeChangeStart`: you want to track successful navigations, not attempted ones
- The page_view event properties:
    - `url`: the full URL of the new page (window.location.href or the URL from the event)
    - `route`: the Next.js route pattern (e.g., `/products/[id]` not `/products/123`)
    - `referrer`: the previous URL (window.location.href before the navigation)
    - `timestamp`: ISO 8601 timestamp of the navigation completion
- Initial page load:
    - `routeChangeComplete` does NOT fire on the initial page load (first server render)
    - To capture the initial page load, fire a page_view event in `useEffect` on the App component's first render

Transitions: You now know how to capture page views. The next lesson implements this in code.

Key Principles:

- Always use `routeChangeComplete`, not `routeChangeStart` — a started navigation that fails or is cancelled would generate a false page view
- Fire an initial page_view in `useEffect` for the first server-rendered page — it won't be caught by `routeChangeComplete`
- Store the previous URL in a ref so you can include it as the referrer in the page_view event

Examples:

- A user who navigates from `/` → `/products` → `/products/123` → `/checkout` should generate exactly 4 page_view events (including the initial `/` load)
- Without capturing `routeChangeComplete`, a 5-step checkout funnel analysis shows only the first page

---

01.2 Track a Page View 💻

Challenge Prompt: Set up page view tracking in a Next.js `_app.js` file. The tracker should fire a `page_view` event on every client-side navigation using the router's `routeChangeComplete` event, and also fire a `page_view` event on initial page load using `useEffect`. Each event should include the current URL, the route pattern, and the timestamp.

Solution Code:

```
import { useEffect, useRef } from 'react';
import { useRouter } from 'next/router';
```

```
import { track } from '../lib/telemetry';

function MyApp({ Component, pageProps }) {
  const router = useRouter();
  const prevUrl = useRef(null);

  useEffect(() => {
    // Capture initial page load
    track('page_view', {
      url: window.location.href,
      route: router.pathname,
      referrer: document.referrer || null,
    });
    prevUrl.current = window.location.href;

    const handleRouteChange = (url) => {
      track('page_view', {
        url: window.location.origin + url,
        route: router.pathname,
        referrer: prevUrl.current,
      });
      prevUrl.current = window.location.origin + url;
    };

    router.events.on('routeChangeComplete', handleRouteChange);
    return () => {
      router.events.off('routeChangeComplete', handleRouteChange);
    };
  }, []);

  return <Component {...pageProps} />;
}

export default MyApp;
```

Challenge Code:

```
import { useEffect, useRef } from 'react';
import { useRouter } from 'next/router';
import { track } from '../lib/telemetry';

function MyApp({ Component, pageProps }) {
  const router = useRouter();
  const prevUrl = useRef(null);

  useEffect(() => {
    // Track the initial page load
    // your code here

    // Set up the route change listener
    const handleRouteChange = (url) => {
```

```
      // Track client-side page views
      // your code here
    };

    router.events.on('routeChangeComplete', handleRouteChange);
    return () => {
      router.events.off('routeChangeComplete', handleRouteChange);
    };
  }, []);

  return <Component {...pageProps} />;
}

export default MyApp;
```

02.0 User Action Events 🕮

Learning Objectives:

- Identify which user actions in a Next.js app are worth tracking as telemetry events
- Apply the event naming convention from the previous course to Next.js-specific actions
- Understand how to attach telemetry to existing event handlers without rewriting them

Content Outline:

- Which user actions to track (the high-value signals):
  - Form submissions: `checkout_submitted`, `search_performed`, `signup_submitted`, `login_attempted`
  - Enrollment or purchase actions: `course_enrolled`, `item_added_to_cart`, `purchase_completed`
  - Content interaction: `video_played`, `content_copied`, `file_downloaded`
  - Navigation intent: `link_clicked` (for important CTAs, not every link)
- Which actions NOT to track:
  - Every button click (too noisy)
  - Every keystroke (too noisy and PII risk)
  - Scroll events (unless scroll depth is a specific product metric)
- How to add telemetry to existing handlers:
  - Don't replace event handlers — augment them
  - Pattern: `const handleSubmit = (e) => { track('checkout_submitted', {...}); existingSubmitLogic(e); }`
  - The `track()` call should be at the START of the handler, before async operations — this ensures the event is captured even if the handler fails later
- Properties to include in user action events:
  - Event-specific context: form field values (sanitized, no PII), item IDs, search queries (truncated/hashed if sensitive)
  - Page context: which page or component did the action occur on?

Transitions: Knowing what to track is half the work. The next lesson implements a specific example.

Key Principles:

- Track actions that connect to product decisions — not every interaction, only the ones that answer a question your team will actually ask
- Put `track()` BEFORE the main logic in handlers — if the handler throws, the telemetry still fires
- Never include raw form input as a telemetry property without scrubbing — form fields often contain PII

Examples:

- A search form submit handler: `track('search_performed', { query: query.trim().substring(0, 100) })` before the search API call
- A course enrollment button: `track('course_enrolled', { course_id: courseId, course_name: courseName })` before the enrollment mutation

---

02.1 Capture a Key Action 💻

Challenge Prompt: Add telemetry to a search form in a Next.js component. When the user submits the search form, fire a `search_performed` event with the sanitized search query (max 100 characters, no PII scrubbing needed for this exercise) and the number of results returned. The telemetry call should happen after the results are fetched so you can include the result count.

Solution Code:

```
import { useState } from 'react';
import { track } from '../lib/telemetry';

export default function SearchPage() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);

  const handleSearch = async (e) => {
    e.preventDefault();
    const sanitizedQuery = query.trim().substring(0, 100);

    const data = await fetch(`/api/search?
q=${encodeURIComponent(sanitizedQuery)}`)
      .then(res => res.json());

    setResults(data.results);

    track('search_performed', {
      query: sanitizedQuery,
      result_count: data.results.length,
    });
  };
```

```
  return (
    <form onSubmit={handleSearch}>
      <input
        value={query}
        onChange={(e) => setQuery(e.target.value)}
        placeholder="Search courses..."
      />
      <button type="submit">Search</button>
      <ul>
        {results.map(r => <li key={r.id}>{r.title}</li>)}
      </ul>
    </form>
  );
}
```

Challenge Code:

```
import { useState } from 'react';
import { track } from '../lib/telemetry';

export default function SearchPage() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);

  const handleSearch = async (e) => {
    e.preventDefault();
    // Sanitize the query
    // your code here

    // Fetch results
    // your code here

    // Track the search event with query and result count
    // your code here
  };

  return (
    <form onSubmit={handleSearch}>
      <input
        value={query}
        onChange={(e) => setQuery(e.target.value)}
        placeholder="Search courses..."
      />
      <button type="submit">Search</button>
      <ul>
        {results.map(r => <li key={r.id}>{r.title}</li>)}
      </ul>
    </form>
  );
}
```

03.0 Errors and Performance 🕮

Learning Objectives:

- Identify the three passive monitoring signals that Next.js makes available
- Understand the difference between intentional user action signals and automatic monitoring signals
- Preview the three sub-topics: error handlers, boundaries, API interceptors, and Web Vitals

Content Outline:

- The distinction between active and passive capture:
  - Active capture (Sections 01-02): you deliberately call `track()` at specific points in your code
  - Passive capture (Section 03): you set up hooks once that fire automatically whenever something happens
- The three passive monitoring signals:
  1. JavaScript errors: `window.onerror` catches unhandled errors; `unhandledrejection` catches unhandled Promise rejections
  2. React rendering errors: Error Boundaries catch errors in component trees that would otherwise crash the app
  3. Performance signals: Web Vitals (LCP, FCP, etc.) and API response times
- Why passive monitoring matters:
  - Users don't report errors — they just leave
  - Performance degradation is invisible to developers unless explicitly measured
  - Error data that includes session context lets you reproduce bugs without asking the user
- Preview: 03.1 covers JavaScript error handlers and React Error Boundaries; 03.2 covers API interceptors and Web Vitals

Transitions: Active signals tell you what users do. Passive signals tell you what's going wrong. The next two lessons implement all three passive monitoring patterns.

Key Principles:

- Passive monitoring doesn't require user interaction — it catches errors that happen between user actions
- Always include `errorId`, `sessionId`, and the recent event history (breadcrumbs) in error telemetry
- React Error Boundaries catch rendering errors; `window.onerror` catches runtime errors — you need both

Examples:

- A JavaScript error in a checkout component that's swallowed by a try-catch but logged to telemetry enables you to identify and fix the bug before it becomes a user-reported incident
- LCP degradation from 2.1s to 3.8s after a deployment is invisible without Web Vitals telemetry

03.1 Error Handlers and Boundaries 🕮

Learning Objectives:

- Implement `window.onerror` and `unhandledrejection` handlers in `_app.js`
- Implement a React Error Boundary that logs errors to telemetry
- Explain what information to include in an error telemetry event

Content Outline:

- Global JavaScript error handlers:
  - `window.onerror = (message, source, line, col, error) => { ... }` — catches synchronous JavaScript errors
  - `window.addEventListener('unhandledrejection', (event) => { ... })` — catches unhandled Promise rejections (async errors)
  - Both should be set up in `useEffect` in `_app.js`
  - Both should call `track('error_occurred', { errorId, type, message, stack (sanitized), ... })`
- The error event properties:
  - `errorId`: a unique ID for this specific error occurrence (UUID or timestamp-based)
  - `type`: `"javascript_error"` or `"unhandled_rejection"` or `"react_render_error"`
  - `message`: the error message (truncated to 500 chars max)
  - `stack`: sanitized stack trace (strip user-specific paths, truncate to 2000 chars)
  - `source`: file name (for window.onerror)
  - `sessionId`: from identity context — links error to user session
- React Error Boundaries:
  - A class component with `static getDerivedStateFromError()` and `componentDidCatch()` methods
  - `componentDidCatch(error, errorInfo)` is called when a child component renders an error
  - Place the Error Boundary at the root in `_app.js`, wrapping `<Component />`
  - Call `track('error_occurred', { type: "react_render_error", message, stack, componentStack })` in `componentDidCatch`
- Stack trace sanitization:
  - Remove absolute file paths (replace with relative paths)
  - Never include `__webpack_require__` internal noise
  - Truncate to 2000 characters maximum

Transitions: Global error handlers and React Error Boundaries cover crashes and exceptions. The next lesson adds performance and API monitoring.

Key Principles:

- `window.onerror` does NOT catch Promise rejections — you need `unhandledrejection` separately
- React Error Boundaries do NOT catch event handler errors — for those, use try-catch + `track()`
- Always sanitize stack traces before sending — they may contain file paths with environment-specific info

Examples:

- A user reports "the checkout is broken" with no other details. With error telemetry, you see `error_occurred { type: "javascript_error", message: "Cannot read property 'price' of undefined", sessionId: "sess_xyz" }` — enough to reproduce and fix immediately
- Without an Error Boundary, a React rendering error causes a blank page with no error event fired — the user sees nothing and you see nothing

---

03.2 API Interceptors and Web Vitals 🕮

Learning Objectives:

- Implement a fetch interceptor that logs API request telemetry
- Implement the `reportWebVitals` export in `_app.js` to capture Core Web Vitals
- Explain what properties to include in API and performance telemetry events

Content Outline:

- API response time monitoring with a fetch interceptor:
  - Wrap the global `fetch` function to intercept all API calls
  - Measure `Date.now()` before and after each request to calculate duration
  - Track `api_call_completed` with: `endpoint` (URL path only, no query params), `method`, `status_code`, `duration_ms`
  - Only intercept calls to your own API (filter by hostname) — don't track third-party CDN calls
  - Place the fetch interceptor setup in `useEffect` in `_app.js` (or as a module-level initialization)
- Web Vitals with `reportWebVitals`:
  - Next.js calls `reportWebVitals(metric)` automatically if you export this function from `_app.js`
  - The `metric` object contains: `name` (LCP, FCP, CLS, FID, TTFB), `value`, `id`, `rating` ('good', 'needs-improvement', 'poor')
  - Track each metric as a `web_vital` event with the metric name, value, and rating
  - Note: LCP, FCP, and TTFB were introduced in Skill 35 as optimization targets — here they're being SENT as telemetry
- The `reportWebVitals` function:

```
export function reportWebVitals(metric) {
  track('web_vital', {
    name: metric.name,
    value: metric.value,
    rating: metric.rating,
  });
}
```

- API timing properties:
  - `endpoint`: the path component only (strip query params and path variables → `/api/courses/[id]`)
  - `method`: GET, POST, etc.
  - `status_code`: the HTTP response status
  - `duration_ms`: milliseconds from request start to response received
  - Do NOT include request/response body — only metadata

Transitions: You now have all three passive monitoring patterns implemented. The next lesson combines everything with a hands-on challenge.

Key Principles:

- Web Vitals values are already being measured by Next.js — you just need to forward them to your telemetry backend via `reportWebVitals`
- Fetch interceptors should NOT log request/response bodies — these often contain sensitive data and violate PII rules
- Normalize endpoint URLs before logging: strip query parameters and replace dynamic segments with placeholders

Examples:

- `reportWebVitals` called with `{ name: "LCP", value: 3800, rating: "poor" }` tells you a page has poor load performance — the same metric you'd optimize in Skill 35, now being measured in production
- A fetch interceptor that logs `{ endpoint: "/api/checkout", duration_ms: 8200, status_code: 500 }` is a backend failure that's now visible in your telemetry, not just server logs

---

03.3 Capture an Error or Performance Signal 💻

Challenge Prompt: Add a global unhandled rejection listener and a `reportWebVitals` export to a Next.js `_app.js` file. The unhandled rejection handler should fire an `error_occurred` telemetry event with the error message and a unique error ID. The `reportWebVitals` export should forward all Web Vitals metrics as `web_vital` events.

Solution Code:

```
import { useEffect } from 'react';
import { track } from '../lib/telemetry';

function MyApp({ Component, pageProps }) {
  useEffect(() => {
    const handleUnhandledRejection = (event) => {
      const errorId =
`err_${Date.now()}_${Math.random().toString(36).substr(2, 5)}`;
      track('error_occurred', {
        errorId,
```

```
        type: 'unhandled_rejection',
        message: event.reason?.message || String(event.reason) || 'Unknown
rejection',
        stack: event.reason?.stack?.substring(0, 2000) || null,
      });
    };

    window.addEventListener('unhandledrejection', handleUnhandledRejection);
    return () => {
      window.removeEventListener('unhandledrejection',
handleUnhandledRejection);
    };
  }, []);

  return <Component {...pageProps} />;
}

export function reportWebVitals(metric) {
  track('web_vital', {
    name: metric.name,
    value: Math.round(metric.value),
    rating: metric.rating,
  });
}

export default MyApp;
```

Challenge Code:

```
import { useEffect } from 'react';
import { track } from '../lib/telemetry';

function MyApp({ Component, pageProps }) {
  useEffect(() => {
    // Set up unhandled rejection listener
    const handleUnhandledRejection = (event) => {
      // Generate a unique error ID and track the error
      // your code here
    };

    window.addEventListener('unhandledrejection', handleUnhandledRejection);
    return () => {
      window.removeEventListener('unhandledrejection',
handleUnhandledRejection);
    };
  }, []);

  return <Component {...pageProps} />;
}

// Export reportWebVitals to capture Web Vitals automatically
```

```
  // your code here

  export default MyApp;
```

04.0 Putting It All Together 🕮

Learning Objectives:

- Describe the complete picture of telemetry capture in a Next.js application
- Explain which `_app.js` exports and hooks handle each capture category
- Identify the gaps that the next learnpack (guardrails) will address

Content Outline:

- The complete `_app.js` telemetry setup:
  1. `useEffect` for router events (page views) + initial page_view
  2. `useEffect` for error handlers (`window.onerror`, `unhandledrejection`)
  3. Error Boundary wrapping `<Component />` for React render errors
  4. Fetch interceptor for API timing
  5. `reportWebVitals` export for Web Vitals
- How the three capture categories map to `_app.js` patterns:
  - Router events → router.events.on in useEffect
  - User actions → per-component event handlers (not in `_app.js` — per action)
  - Passive monitoring → window listeners, Error Boundary, reportWebVitals in `_app.js`
- What's NOT captured yet:
  - Consent gate: the next learnpack will add a consent check before any event is fired
  - PII scrubbing: some events (search queries, form submissions) may contain sensitive data that needs sanitization
  - Allowlists: the telemetry system sends whatever properties you pass — the next learnpack adds constraints
  - Sampling: high-frequency signals (scroll depth, API calls) should be sampled, not sent on every occurrence
- The handoff to the next learnpack: these four guardrails complete the telemetry implementation

Transitions: Test your understanding before moving to the guardrails learnpack.

Key Principles:

- `_app.js` is the telemetry control center for passive signals; individual components handle user action signals
- Three well-placed hooks cover the majority of signals a production app needs
- Capture without guardrails is a compliance and privacy risk — the next learnpack is not optional

Examples:

- A complete `_app.js` with all five telemetry hooks (router, onerror, unhandledrejection, ErrorBoundary, reportWebVitals) is under 50 lines of additional code

- A team that ships the capture hooks without the consent gate is collecting data from users who haven't agreed to it — a legal problem in most jurisdictions

---

04.1 Next.js Telemetry Capture 🫣

Learning Objectives:

- Apply the three capture patterns to a new Next.js scenario
- Identify which hook belongs at which location in a Next.js application
- Diagnose missing telemetry from a described implementation

Question Format: Scenario-based

Questions:

1. A developer adds a `track('page_view', ...)` call inside the `useEffect` of each individual page component in their Next.js app. What is wrong with this approach compared to using `routeChangeComplete` in `_app.js`? What specific telemetry events might be missing?

2. A Next.js app fires error telemetry using only `window.onerror`. A React component crashes during rendering because of a null reference in JSX. Does `window.onerror` catch this? What is missing, and how do you fix it?

3. Your team reports that Web Vitals metrics are never appearing in your analytics dashboard, even though you have a telemetry sender set up. You added the `track('web_vital', ...)` call inside a `useEffect` in `_app.js`. What is the likely cause and the correct fix?

4. A checkout form component needs to track two events: `form_focused` (when any field is focused) and `checkout_submitted` (when the form is successfully submitted). Which of these is a high-value signal worth tracking? Which violates the "only track signals that answer product questions" rule? Explain your reasoning.

5. A developer wants to track API response times. They add `console.log` statements before and after every `fetch()` call across 30 different components. What is the problem with this approach, and how would you implement a fetch interceptor in `_app.js` instead?

Evaluation Criteria:

- Q1: Does the student identify that `useEffect` fires on every render of a page, potentially double-counting, AND that client-side navigation between pages doesn't trigger a new page component mount in Next.js (so some navigations would be missed)?
- Q2: Does the student correctly identify that `window.onerror` does NOT catch React render errors, and that a React Error Boundary is required?
- Q3: The student should recognize that `reportWebVitals` is a special Next.js export — it must be exported from `_app.js` as a named export, not called inside a `useEffect`
- Q4: `checkout_submitted` is high-value (answers "how many users complete checkout?"); `form_focused` is noise (doesn't answer any product question and fires on every focus event)
- Q5: Per-component console.log is unmaintainable and not automated; a fetch interceptor in `_app.js` wraps the global `fetch` once and covers all components automatically

Score Interpretation:

- 4-5 correct: Strong grasp of Next.js-specific telemetry capture patterns
- 2-3 correct: Good foundation; review the difference between global hooks and per-component handlers
- 0-1 correct: Return to the three capture categories and their corresponding Next.js patterns

---

05.0 Outro

Content Outline:

- Course recap: what students accomplished
  - Implemented page view tracking using `routeChangeComplete` and initial `useEffect` in `_app.js`
  - Added user action telemetry to a search form handler with proper sanitization
  - Set up passive monitoring: `window.onerror`, `unhandledrejection`, React Error Boundary, fetch interceptor, and `reportWebVitals`
  - Identified the four guardrails that the next learnpack will add
- Connection to bigger picture
  - This course completes the capture layer: events now flow from three capture points into the batch sender, then to the backend
  - The next learnpack (Guardrails) adds the safety layer: consent checks, property allowlists, PII scrubbing, and dev/prod separation
  - After guardrails, the full telemetry pipeline is production-ready
- Next steps and continued learning
  - Next.js Analytics docs: how Next.js itself uses telemetry internally
  - web-vitals npm package: the same library Next.js uses internally for `reportWebVitals`
  - Sentry's Next.js integration: a production-grade error monitoring library that uses all three capture patterns
- Encouragement
  - Most teams instrument their apps reactively — after a bug is reported. You're building proactive visibility before problems occur.

Note: This is conclusion only — no theory, exercises, or assessment.